

Tuning Report

1. Data Preparation and Cleaning:

- Loading ratings and book data.
- Cleaning text fields (title, author, and publisher) by converting to lowercase and removing punctuation.
- Creating a combined field for content-based filtering using book title, author, and publisher.
- Encoding user and book IDs as integers for matrix indexing.

2. Collaborative Filtering (KNN) Model:

- Mapped user and book IDs to integer indices (user_idx, book_idx) for building matrices.
- Built a user-item interaction matrix of shape (77805 users $\times$ 185973 books) with ratings filled in.
- This matrix is used in collaborative filtering algorithms.
- Using NearestNeighbors with the cosine similarity metric to find similar users based on their ratings.
- Hyperparameter Tuning: You are optimizing parameters like K (number of nearest neighbors) and similarity threshold by using a grid search approach. You evaluate precision at K for each parameter setting.
- Train-Test Split: The interaction matrix is split into a training and test set using a sparse matrix format for efficiency.

Parameters Tuned:

- K (Number of Neighbors): [5, 10, 20]
- Threshold (Minimum similarity for recommendation): [0.1, 0.2]

Algorithm Used:

- `sklearn.neighbors.NearestNeighbors`
- `metric='cosine'`
- `algorithm='brute'`
- `n_jobs=-1` (for parallel processing)
- **Evaluation Metric:** Precision@5
- **Results:**

K	Threshold	Precision@5
5	0.1	0.2044 (Best)
5	0.2	0.1967
10	0.1	0.1900
10	0.2	0.1744
20	0.1	0.1764
20	0.2	0.1403

Best Parameters:

- $K = 5$
- Threshold = 0.1
- **Precision@5 = 0.2044**

4. Content-Based Filtering (CB) Model:

- **TF-IDF Vectorization:** The content-based model uses `TfidfVectorizer` to convert book titles, authors, and publishers into a numerical representation of text features (n-grams).
- **Cosine Similarity:** You calculate item-item similarities based on these text features to recommend books similar to those a user has already rated.

- **Hyperparameter Tuning:** The parameters tuned include:
 - ngram_range: (1,1) and (1,2)
 - max_features: 1000 and 5000
 - min_df: 2 (ignore rare words)
 - max_df: 0.8 (ignore too frequent words)

You constructed a combined column using title, author, and publisher for each book and used TfidfVectorizer for vectorization.

N-gram	Max Features	Precision@5
(1,1)	1000	0.0096
(1,1)	5000	0.0180
(1,2)	1000	0.0100
(1,2)	5000	0.0191 (Best)

Best Parameters:

- ngram_range = (1, 2)
- max_features = 5000
- min_df = 2
- max_df = 0.8
- **Precision@5 = 0.0191**

1. Summary of Parameters Tuned

a. Collaborative Filtering (User-User CF)

- **K (Number of Neighbors):** [5, 10, 20]
- **Threshold** (minimum similarity for neighbor inclusion): [0.1, 0.2]
- **Metric:** Precision@5

b. Content-Based Filtering (CBF)

- **TF-IDF Parameters Tuned:**
 - `ngram_range`: (1,1) and (1,2)
 - `max_features`: 1000, 5000
 - `min_df`: 2 (exclude rare terms)
 - `max_df`: 0.8 (exclude overly common terms)
- **Metric**: Precision@5

2. What Worked and What Didn't

a. Collaborative Filtering

- **Worked:**
 - **K = 5** performed best across all threshold values.
 - **Lower threshold (0.1)** retained more neighbors, improving recommendation quality.
- **Didn't Work:**
 - Increasing **K** beyond 5 introduced less similar neighbors, reducing precision.
 - **Higher threshold (0.2)** filtered out too many neighbors, harming performance.

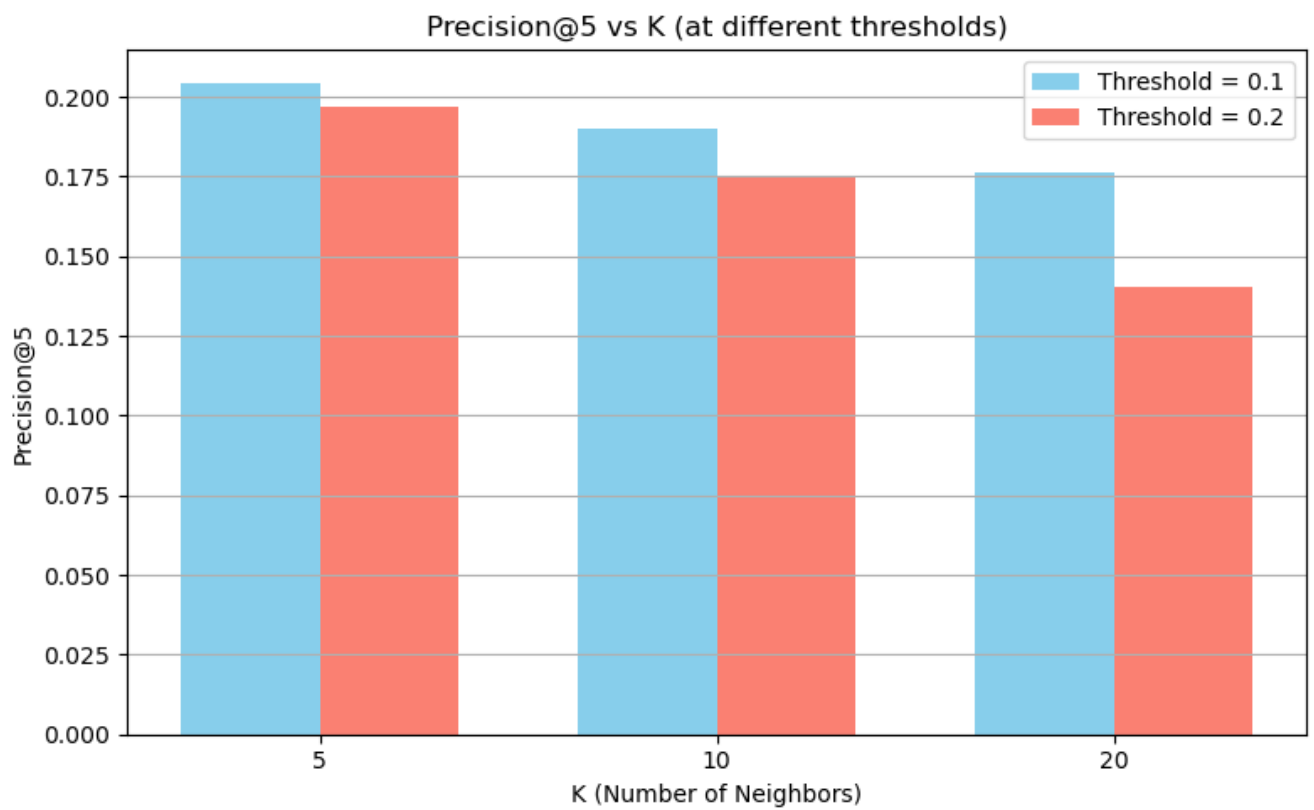
b. Content-Based Filtering

- **Worked:**
 - **ngram_range = (1,2)** captured more context, boosting similarity accuracy.
 - **max_features = 5000** retained more vocabulary, improving coverage.
- **Didn't Work:**
 - **ngram_range = (1,1)** and **low max_features (1000)** led to underfitting.
 - Small feature set limited meaningful content similarity.

3. Visual Comparison of Configurations

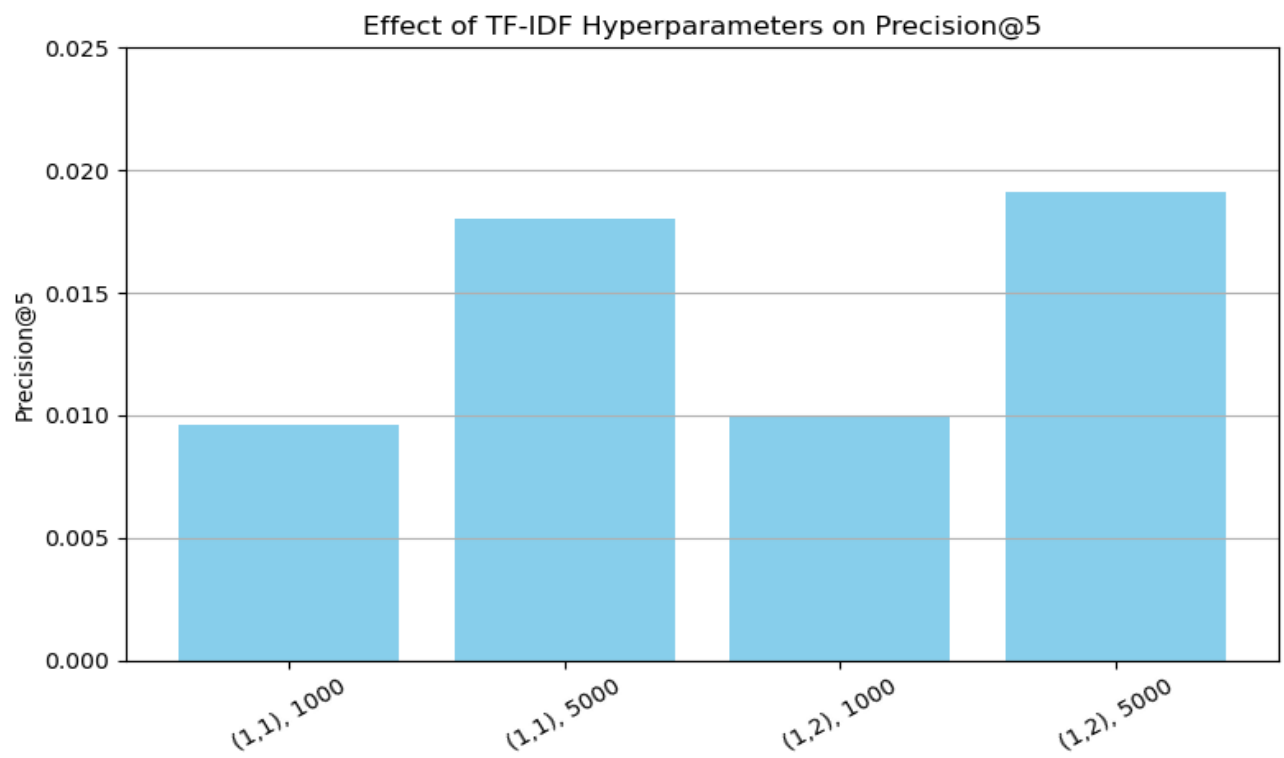
a. Collaborative Filtering (User-User CF)

K	Threshold	Precision@5
5	0.1	0.2044 (Best)
5	0.2	0.1967
10	0.1	0.1900
10	0.2	0.1744
20	0.1	0.1764
20	0.2	0.1403



b. Content-Based Filtering (TF-IDF)

N-gram	Max Features	Precision@5
(1,1)	1000	0.0096
(1,1)	5000	0.0180
(1,2)	1000	0.0100
(1,2)	5000	0.0191 (Best)



4. Final Selection Rationale

a. CF (User-User)

- **Best config:** $K = 5$, threshold = 0.1
- **Precision@5:** 0.2044 (highest across all models)

- Selected for its high **accuracy** and **personalized recommendations**

b. CBF (TF-IDF)

- **Best config:** ngram_range = (1,2), max_features = 5000
- **Precision@5:** 0.0191
- Selected for better **coverage** and ability to recommend for **cold-start users**

5. Manual Observations

a. CF (User-User)

- **Strengths:**
 - Excellent personalization
 - High accuracy when user has sufficient history
- **Weaknesses:**
 - **Very low coverage** (cold-start users get no recommendations)
 - **Sparse matrix** limits effectiveness as K increases

b. CBF (TF-IDF)

- **Strengths:**
 - Works well for **new users/items** using metadata
 - Better **coverage** and **diversity**
- **Weaknesses:**
 - Lower precision due to reliance on textual similarity.
 - Does not capture user preferences from behavior.

6. Performance Metrics Comparison

a. Collaborative Filtering (CF, Best Config):

- Precision@5: 0.2044
- Coverage: ~0.00
- Diversity: 1.0000
- Hit Rate: 0.3739
- **Cold-Start Capability:** Not suitable
- Very accurate, but only for users/items in the training data (low coverage).

b. Content-Based Filtering (CBF, Best Config):

- Precision@5: 0.0191
- Coverage: 1.8502
- Diversity: 0.4391
- Hit Rate: 0.0756
- **Cold-Start Capability:** Suitable
- Broader coverage and diversity, useful for cold-start problems (new users/items).